

Steps to implement recursion function :

- 1) Implement the given task, as a normal function using while loop.
- 2) Inside the function, if any variables are declared and initialized, then split that declaration into 2 partitions.
 - i) declaration as a formal parameter to the function definition.
 - ii) initialization as a passing value from the function call to the function definition.
- 3) replace while with if control statement.
- 4) place a recursion call as a last instruction inside the if block with as usual formal parameters.
- 5) place a return keyword before the recursion call if the function return type is non-void.
- 6) if any instructions are there after if block, then keep it inside the else part.
- 7) modify the function declaration based on function definition.

Optional steps

- 8) reduce the logic instructions, with possible substitutions.
- 9) check, if any possible substitutions for formal parameters inside the recursion call.
(and then remove the substitute instructions.)

Reversing the number as a normal function:

```
#include<stdio.h>
int rev_num(int);
int main()
{
    int num;
    printf("Enter the number\n");
    scanf("%d",&num);

    int rev = rev_num(num);
    printf("reverse num = %d\n",rev);
}
int rev_num(int num)
{
    int rev = 0,rem;
    while(num != 0)
    {
        rem = num%10;
        rev = rev*10+rem;
        num = num/10;
    }
    return rev;
}
```

Reversing the number using recursion function:

```
#include<stdio.h>
int rev_num(int,int);
int main()
{
    int num;
    printf("Enter the number\n");
    scanf("%d",&num);

    int rev = rev_num(num,0);
    printf("reverse num = %d\n",rev);
}
int rev_num(int num,int rev)
{
    int rem;
    if(num != 0)
        return rev_num(num/10,rev*10+(num%10))
    else
        return rev;
}

int rev_num(int num,int rev)
{
    int rem;
    if(num != 0)
    {
        rev = rev*10+(num%10);
        num = num/10;
        return rev_num(num,rev);
    }
    else
        return rev;
}
```

fibonacci normal function :

```
#include<stdio.h>
void fibo(int);
int main()
{
    int N;
    printf("Enter the N value\n");
    scanf("%d",&N);

    fibo(N);
    printf("\n");
}
void fibo(int N)
{
    int x = 0, y = 1, z;
    while(x <= N)
    {
        printf("%d ",x);
        z = x+y;
        x = y;
        y = z;
    }
}
```

fibonacci recursion function :

```
#include<stdio.h>
void fibo(int,int,int);
int main()
{
    int N;
    printf("Enter the N value\n");
    scanf("%d",&N);

    fibo(N,0,1);
    printf("\n");
}
void fibo(int N,int x,int y)
{
    int z;
    if(x <= N)
    {
        printf("%d ",x);
        z = x+y;
        x = y;
        y = z;
        fibo(N,x,y);
    }
}
```



```
void fibo(int N,int x,int y)
{
    int z;
    if(x <= N)
    {
        printf("%d ",x);
        x = y;
        y = x+y;
        fibo(N,x,y);
    }
}
```

```
void fibo(int N,int x,int y)
{
    int z;
    if(x <= N)
    {
        printf("%d ",x);
        fibo(N,y,x+y);
    }
}
```

```
void fibo(int N,int x,int y)
{
    int z;
    if(x <= N)
        printf("%d ",x),fibo(N,y,x+y);
}
```

Important assignments :

- 1) implement recursion function for factorial num.
- 2) implement recursion function for reversing the string.
- 3) WAP to implement recursion function to check the given number is prime number or not.

Hint : if the num is prime, then return 0
if the num is not prime, then return -1.
from the recursion function.